ListaDwukierunkowa

Generated by Doxygen 1.18.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 DoublyLinkedList Class Reference . . . 5

3.1.1 Detailed Description . . . 6

3.1.2 Member Function Documentation . . . 6

3.1.2.1 begin() . . . 6

3.1.2.2 end() . . . 6

3.1.2.3 pop_wherever() . . . 6

3.1.2.4 push_back() . . . 7

3.1.2.5 push_front() . . . 7

3.1.2.6 push_wherever() . . . 7

3.2 Factory Class Reference . . . 7

3.2.1 Detailed Description . . . 8

3.2.2 Member Function Documentation . . . 8

3.2.2.1 createNode() . . . 8

3.3 Iterator Class Reference . . . 8

3.3.1 Detailed Description . . . 9

3.3.2 Constructor & Destructor Documentation . . . 9

3.3.2.1 Iterator() . . . 9

3.3.3 Member Function Documentation . . . 9

3.3.3.1 operator"!=() . . . 9

3.3.3.2 operator∗() . . . 10

3.3.3.3 operator++() . . . 10

3.3.3.4 operator--() . . . 10

3.4 Node Class Reference . . . 11

3.4.1 Detailed Description . . . 11

3.4.2 Constructor & Destructor Documentation . . . 11

3.4.2.1 Node() . . . 11

4 File Documentation 13

4.1 DoublyLinkedList.h . . . 13

4.2 ListaDwukierunkowa_Pyrdol_Jakub/Factory.cpp File Reference . . . 13

4.2.1 Detailed Description . . . 14

4.3 Factory.h . . . 14

4.4 ListaDwukierunkowa_Pyrdol_Jakub/Iterator.cpp File Reference . . . 14

4.4.1 Detailed Description . . . 14

4.5 Iterator.h . . . 14

4.6 ListaDwukierunkowa_Pyrdol_Jakub/Node.cpp File Reference . . . 14

4.6.1 Detailed Description 15
4.7 Node.h 15

Index **17**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DoublyLinkedList
: Główna klasa zarządzająca listą dwukierunkową 5

Factory
: Klasa implementująca wzorzec Factory 7

Iterator
: Klasa implementująca wzorzec Iterator dla listy dwukierunkowej 8

Node
: Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej 11

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

ListaDwukierunkowa_Pyrdol_Jakub/DoublyLinkedList.h 13
ListaDwukierunkowa_Pyrdol_Jakub/Factory.cpp
    Implementacja metod klasy Factory 13
ListaDwukierunkowa_Pyrdol_Jakub/Factory.h 14
ListaDwukierunkowa_Pyrdol_Jakub/Iterator.cpp
    Implementacja metod klasy Iterator 14
ListaDwukierunkowa_Pyrdol_Jakub/Iterator.h 14
ListaDwukierunkowa_Pyrdol_Jakub/Node.cpp
    Plik zawierający implementację metod klasy Node 14
ListaDwukierunkowa_Pyrdol_Jakub/Node.h 15

Chapter 3

Class Documentation

3.1 DoublyLinkedList Class Reference

Główna klasa zarządzająca listą dwukierunkową.

`#include <DoublyLinkedList.h>`

Public Member Functions

- void push_front (int value)
  Dodaje nowy element na sam początek listy.
- void push_back (int value)
  Dodaje nowy element na sam koniec listy.
- void push_wherever (int index, int value)
  Wstawia nowy element pod wskazany indeks.
- void **pop_front** ()
  Usuwa pierwszy element z listy i zwalnia jego pamięć.
- void **pop_back** ()
  Usuwa ostatni element z listy i zwalnia jego pamięć.
- void pop_wherever (int index)
  Usuwa element znajdujący się pod wskazanym indeksem.
- void **display_list** ()
  Wyświetla zawartość całej listy od początku do końca.
- void **display_reversed_list** ()
  Wyświetla zawartość całej listy w odwrotnej kolejności (od końca do początku).
- void **display_next_element** ()
  Wyświetla następny element w sekwencji.
- void **display_prev_element** ()
  Wyświetla poprzedni element w sekwencji.
- void **clear_list** ()
  Usuwa wszystkie elementy z listy, całkowicie ją czyszcząc i zwalniając pamięć.
- Iterator begin ()
  Zwraca iterator ustawiony na początek listy.
- Iterator end ()
  Zwraca iterator reprezentujący koniec listy.

3.1.1 Detailed Description

Główna klasa zarządzająca listą dwukierunkową.

Klasa przechowuje wskaźniki na początek i koniec listy. Zarządza dodawaniem, usuwaniem i wyświetlaniem elementów, wykorzystując do tego wzorzec Factory oraz wydając iteratory (wzorzec Iterator).

3.1.2 Member Function Documentation

3.1.2.1 begin()

```
Iterator DoublyLinkedList::begin ()
```

Zwraca iterator ustawiony na początek listy.

Returns

Iterator wskazujący na element head.

3.1.2.2 end()

```
Iterator DoublyLinkedList::end ()
```

Zwraca iterator reprezentujący koniec listy.

Returns

Iterator ustawiony za ostatnim elementem (najczęściej wskazujący na nullptr).

3.1.2.3 pop_wherever()

```
void DoublyLinkedList::pop_wherever (
            int index)
```

Usuwa element znajdujący się pod wskazanym indeksem.

Parameters

index	Pozycja elementu do usunięcia (liczona od 0).

3.1.2.4 push_back()

```
void DoublyLinkedList::push_back (
            int value)
```

Dodaje nowy element na sam koniec listy.

Parameters

value	Wartość liczbowa do zapisania w nowym węźle.

3.1.2.5 push_front()

```
void DoublyLinkedList::push_front (
            int value)
```

Dodaje nowy element na sam początek listy.

Parameters

value	Wartość liczbowa do zapisania w nowym węźle.

3.1.2.6 push_wherever()

```
void DoublyLinkedList::push_wherever (
            int index,
            int value)
```

Wstawia nowy element pod wskazany indeks.

Parameters

index	Pozycja, na którą ma zostać wstawiony element (liczona od 0).
value	Wartość liczbowa do zapisania w nowym węźle.

The documentation for this class was generated from the following file:

- ListaDwukierunkowa_Pyrdol_Jakub/DoublyLinkedList.h

3.2 Factory Class Reference

Klasa implementująca wzorzec Factory.

```
#include <Factory.h>
```

Public Member Functions

- Node ∗ createNode (int value)

  Tworzy nowy obiekt węzła na stercie.

3.2.1 Detailed Description

Klasa implementująca wzorzec Factory.

Odpowiada za bezpieczne tworzenie nowych węzłów (Node) na stercie. Zastosowanie tego wzorca pozwala na oddzielenie logiki alokacji pamięci od głównej logiki zarządzania samą listą dwukierunkową.

3.2.2 Member Function Documentation

3.2.2.1 createNode()

```
Node * Factory::createNode (
            int value)
```

Tworzy nowy obiekt węzła na stercie.

Alokuje dynamicznie pamięć dla nowego węzła i zwraca na niego wskaźnik.

Wykorzystuje operator new do alokacji pamięci dla nowego węzła i inicjuje go przekazaną wartością.

Parameters

value	Wartość całkowita liczby, która zostanie przypisana do nowego węzła.

Returns

Node∗ Wskaźnik na nowo utworzony węzeł.

The documentation for this class was generated from the following files:

- ListaDwukierunkowa_Pyrdol_Jakub/Factory.h
- ListaDwukierunkowa_Pyrdol_Jakub/Factory.cpp

3.3 Iterator Class Reference

Klasa implementująca wzorzec Iterator dla listy dwukierunkowej.

```
#include <Iterator.h>
```

Public Member Functions

- Iterator (Node ∗current)

  Konstruktor inicjujący iterator.
- void operator++ ()

  Operator inkrementacji.
- void operator-- ()

  Operator dekrementacji.
- int operator∗ ()

  Operator wyłuskania.
- bool operator!= (const Iterator &other)

  Operator nierówności.

3.3.1 Detailed Description

Klasa implementująca wzorzec Iterator dla listy dwukierunkowej.

Pozwala przechodzenie przez elementy listy w obu kierunkach za pomocą standardowych operatorów, ukrywając przed użytkownikiem logikę wskaźników.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 Iterator()

```
Iterator::Iterator (
            Node * current)
```

Konstruktor inicjujący iterator.

Przypisuje wskaźnik na węzeł początkowy do zmiennej current.

Parameters

current	Wskaźnik na węzeł startowy, od którego iterator rozpocznie pracę.

3.3.3 Member Function Documentation

3.3.3.1 operator"!=()

```
bool Iterator::operator!= (
            const Iterator & other)
```

Operator nierówności.

Sprawdza, czy węzły wskazywane przez dwa iteratory są od siebie różne.

Porównuje dwa iteratory. W pętlach, sprawdza czy iterator nie dotarł jeszcze do końca kolekcji.

Parameters

other	Inny obiekt iteratora, z którym porównujemy obecny.

Returns

true Jeśli iteratory wskazują na różne węzły.

false Jeśli iteratory wskazują na ten sam węzeł.

3.3.3.2 operator∗()

```
int Iterator::operator* ()
```

Operator wyłuskania.

Zwraca wartość z węzła wskazywanego przez iterator.

Returns

Liczba całkowita przechowywana w aktualnie wskazywanym węźle.

3.3.3.3 operator++()

```
void Iterator::operator++ ()
```

Operator inkrementacji.

Przesuwa iterator na kolejny element za pomocą wskaźnika next.

Przesuwa wskaźnik iteratora na następny element listy.

3.3.3.4 operator--()

```
void Iterator::operator-- ()
```

Operator dekrementacji.

Cofa iterator na poprzedni element za pomocą wskaźnika prev.

Przesuwa wskaźnik iteratora na poprzedni element listy.

The documentation for this class was generated from the following files:

- ListaDwukierunkowa_Pyrdol_Jakub/Iterator.h
- ListaDwukierunkowa_Pyrdol_Jakub/Iterator.cpp

3.4 Node Class Reference

Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej.

`#include <Node.h>`

Public Member Functions

- Node (int value)

  Konstruktor inicjujący nowy węzeł.

Public Attributes

- int **value**

  Przechowywana wartość całkowitą.
- Node ∗ **next**

  Wskaźnik na następny element na liście.
- Node ∗ **prev**

  Wskaźnik na poprzedni element na liście.

3.4.1 Detailed Description

Klasa reprezentująca pojedynczy węzeł listy dwukierunkowej.

Służy jako podstawowy kontener przechowujący dane (liczbę całkowitą) oraz informacje o sąsiadujących elementach w strukturze listy.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 Node()

```
Node::Node (
            int value)
```

Konstruktor inicjujący nowy węzeł.

Tworzy nowy węzeł z zadaną wartością.

Parameters

value	Wartość początkowa przypisywana do zmiennej value.

Inicjuje węzeł ustawiając jego wartość na tę przekazaną w parametrze. Wskaźniki next i prev są bezpiecznie ustawiane na nullptr, ponieważ świeżo utworzony węzeł nie jest jeszcze połączony z resztą listy.

Parameters

value	Wartość liczbowa do przechowania w węźle.

The documentation for this class was generated from the following files:

- ListaDwukierunkowa_Pyrdol_Jakub/Node.h
- ListaDwukierunkowa_Pyrdol_Jakub/Node.cpp

Chapter 4

File Documentation

4.1 DoublyLinkedList.h

```
00001 #ifndef LISTADWUKIERUNKOWA_PYRDOL_JAKUB_DOUBLYLINKEDLIST_H
00002 #define LISTADWUKIERUNKOWA_PYRDOL_JAKUB_DOUBLYLINKEDLIST_H
00003 #include "Factory.h"
00004 #include "Iterator.h"
00005 #include "Node.h"
00006
00015 class DoublyLinkedList
00016 {
00017 private:
00021     Node* head;
00022
00026     Node* tail;
00027
00031     Factory factory;
00032
00036     int size;
00037
00038 public:
00044     void push_front(int value);
00045
00051     void push_back(int value);
00052
00059     void push_wherever(int index, int value);
00060
00064     void pop_front();
00065
00069     void pop_back();
00070
00076     void pop_wherever(int index);
00077
00081     void display_list();
00082
00086     void display_reversed_list();
00087
00091     void display_next_element();
00092
00096     void display_prev_element();
00097
00101     void clear_list();
00102
00108     Iterator begin();
00109
00115     Iterator end();
00116 };
00117
00118 #endif //LISTADWUKIERUNKOWA_PYRDOL_JAKUB_DOUBLYLINKEDLIST_H
```

4.2 ListaDwukierunkowa_Pyrdol_Jakub/Factory.cpp File Reference

Implementacja metod klasy Factory.

```
#include "Factory.h"
```

4.2.1 Detailed Description

Implementacja metod klasy Factory.

4.3 Factory.h

```
00001 #ifndef LISTADWUKIERUNKOWA_PYRDOL_JAKUB_FACTORY_H
00002 #define LISTADWUKIERUNKOWA_PYRDOL_JAKUB_FACTORY_H
00003 #include "Node.h"
00004
00012 class Factory
00013 {
00014 public:
00024     Node* createNode(int value);
00025 };
00026
00027 #endif //LISTADWUKIERUNKOWA_PYRDOL_JAKUB_FACTORY_H
```

4.4 ListaDwukierunkowa_Pyrdol_Jakub/Iterator.cpp File Reference

Implementacja metod klasy Iterator.

```
#include "Iterator.h"
```

4.4.1 Detailed Description

Implementacja metod klasy Iterator.

4.5 Iterator.h

```
00001 #ifndef LISTADWUKIERUNKOWA_PYRDOL_JAKUB_ITERATOR_H
00002 #define LISTADWUKIERUNKOWA_PYRDOL_JAKUB_ITERATOR_H
00003 #include "Node.h"
00004
00011 class Iterator
00012 {
00013 private:
00017     Node* current;
00018
00019 public:
00025     Iterator(Node* current);
00026
00032     void operator++();
00033
00039     void operator--();
00040
00046     int operator*();
00047
00058     bool operator!=(const Iterator& other);
00059 };
00060
00061 #endif //LISTADWUKIERUNKOWA_PYRDOL_JAKUB_ITERATOR_H
```

4.6 ListaDwukierunkowa_Pyrdol_Jakub/Node.cpp File Reference

Plik zawierający implementację metod klasy Node.

```
#include "Node.h"
```

4.6.1 Detailed Description

Plik zawierający implementację metod klasy Node.

4.7 Node.h

```
00001 #ifndef LISTADWUKIERUNKOWA_PYRDOL_JAKUB_NODE_H
00002 #define LISTADWUKIERUNKOWA_PYRDOL_JAKUB_NODE_H
00003
00010 class Node
00011 {
00012 public:
00016     int value;
00017
00021     Node* next;
00022
00026     Node* prev;
00027
00033     Node(int value);
00034 };
00035
00036 #endif
```

Index

begin
    DoublyLinkedList, 6

createNode
    Factory, 8

DoublyLinkedList, 5
    begin, 6
    end, 6
    pop_wherever, 6
    push_back, 6
    push_front, 7
    push_wherever, 7

end
    DoublyLinkedList, 6

Factory, 7
    createNode, 8

Iterator, 8
    Iterator, 9
    operator!=, 9
    operator++, 10
    operator--, 10
    operator∗, 10

ListaDwukierunkowa_Pyrdol_Jakub/DoublyLinkedList.h, 13
ListaDwukierunkowa_Pyrdol_Jakub/Factory.cpp, 13
ListaDwukierunkowa_Pyrdol_Jakub/Factory.h, 14
ListaDwukierunkowa_Pyrdol_Jakub/Iterator.cpp, 14
ListaDwukierunkowa_Pyrdol_Jakub/Iterator.h, 14
ListaDwukierunkowa_Pyrdol_Jakub/Node.cpp, 14
ListaDwukierunkowa_Pyrdol_Jakub/Node.h, 15

Node, 11
    Node, 11

operator!=
    Iterator, 9
operator++
    Iterator, 10
operator--
    Iterator, 10
operator∗
    Iterator, 10

pop_wherever
    DoublyLinkedList, 6
push_back
    DoublyLinkedList, 6
push_front
    DoublyLinkedList, 7
push_wherever
    DoublyLinkedList, 7